

Qwen3-Coder iOS 界面性能优化：从代码实现看经典案例



案例一：优化 UITableView 的复杂 Cell

优化前：低效代码（Objective-C）

Code block

```
1  // MyTableViewCell.h
2  @property (weak, nonatomic) IBOutlet UILabel *titleLabel;
3  @property (weak, nonatomic) IBOutlet UILabel *subtitleLabel;
4  @property (weak, nonatomic) IBOutlet UIImageView *iconImageView;
5  @property (weak, nonatomic) IBOutlet UIView *containerView; // 一个包含多个子视图
    的容器
6
7  // MyTableViewCell.m
8  - (void)layoutSubviews {
9      [super layoutSubviews];
10
11      // ❌ 错误示范：在 layoutSubviews 里做复杂计算
12      CGSize titleSize = [self.titleLabel.text
13      boundingRectWithSize:CGSizeMake(self.bounds.size.width - 40, CGFLOAT_MAX)
14
15      options:NSStrngDrawingUsesLineFragmentOrigin
16
17      attributes:@{NSFontAttributeName: self.titleLabel.font}
18
19      context:nil].size;
20
21      // 重新计算所有子视图的位置
22      self.titleLabel.frame = CGRectMake(20, 10, titleSize.width,
23      titleSize.height);
24      self.subtitleLabel.frame = CGRectMake(20, 10 + titleSize.height + 5,
25      self.bounds.size.width - 40, 20);
26      self.iconImageView.frame = CGRectMake(self.bounds.size.width - 50, 10, 30,
27      30);
28
29      // 假设有 5 个子视图，这里要计算 5 次
```

```

23     for (UIView *subview in self.containerView.subviews) {
24         // ... 更多复杂计算
25     }
26 }
27
28 // 在 Controller 里
29 - (UITableViewCell *)tableView:(UITableView *)tableView cellForRowAtIndexPath:
(NSIndexPath *)indexPath {
30     MyTableViewCell *cell = [tableView
dequeueReusableCellWithIdentifier:@"MyCell" forIndexPath:indexPath];
31
32     MyModel *model = self.dataArray[indexPath.row];
33
34     // ❌ 错误示范: 直接设置图片, 可能触发主线程解码
35     cell.titleLabel.text = model.title;
36     cell.subtitleLabel.text = model.subtitle;
37     cell.iconImageView.image = [UIImage imageNamed:model.iconName]; //
imageNamed 也可能有性能问题
38
39     return cell;
40 }
41
42 - (CGFloat)tableView:(UITableView *)tableView heightForRowAtIndexPath:
(NSIndexPath *)indexPath {
43     // ❌ 错误示范: 每次都要重新计算高度
44     MyModel *model = self.dataArray[indexPath.row];
45     CGSize titleSize = [model.title
boundingRectWithSize:CGSizeMake(SCREEN_WIDTH - 40, CGFLOAT_MAX)
46
options:NSStrngDrawingUsesLineFragmentOrigin
47                                     attributes:@{NSFontAttributeName:
[UIFont systemFontOfSize:16]}
48                                     context:nil].size;
49     // ... 计算更多内容的高度
50     return 10 /* top padding */ + titleSize.height + 5 /* spacing */ + 20 /*
subtitle height */ + 10 /* bottom padding */;
51 }

```

优化后：高效代码（Objective-C）

Code block

```

1 // MyModel.h (在 Model 里预存布局信息)
2 @interface MyModel : NSObject
3 @property (nonatomic, copy) NSString *title;
4 @property (nonatomic, copy) NSString *subtitle;

```

```

5  @property (nonatomic, copy) NSString *iconName;
6  @property (nonatomic, assign) CGFloat cellHeight; // 预计算好的高度
7  @property (nonatomic, assign) CGRect titleLabelFrame; // 预计算好的 frame
8  @property (nonatomic, assign) CGRect subtitleLabelFrame;
9  @property (nonatomic, assign) CGRect iconImageViewFrame;
10 @end
11
12 // 在 Controller 里，数据请求回来后，立刻进行预计算
13 - (void)onDataReceived:(NSArray<MyModel *> *)data {
14     dispatch_async(dispatch_get_global_queue(DISPATCH_QUEUE_PRIORITY_DEFAULT,
15     0), ^{
16         for (MyModel *model in data) {
17             // 在后台线程预计算所有布局信息
18             CGSize titleSize = [model.title
19 boundingRectWithSize:CGSizeMake(SCREEN_WIDTH - 40, CGFLOAT_MAX)
20 options:NSStrngDrawingUsesLineFragmentOrigin
21 attributes:@{NSFontAttributeName: [UIFont systemFontOfSize:16]}
22             context:nil].size;
23             model.cellHeight = 10 + titleSize.height + 5 + 20 + 10;
24             model.titleLabelFrame = CGRectMake(20, 10, titleSize.width,
25 titleSize.height);
26             model.subtitleLabelFrame = CGRectMake(20, 10 + titleSize.height +
27 5, SCREEN_WIDTH - 40, 20);
28             model.iconImageViewFrame = CGRectMake(SCREEN_WIDTH - 50, 10, 30,
29 30);
30         }
31
32         dispatch_async(dispatch_get_main_queue(), ^{
33             // 回到主线程更新 UI
34             self.dataArray = data;
35             [self.tableView reloadData];
36         });
37     });
38 }
39
40 // MyTableViewCell.m (优化后的 Cell)
41 - (void)layoutSubviews {
42     [super layoutSubviews];
43     // ✅ 优化：直接使用 Model 里预计算好的 frame
44     MyModel *model = self.model; // 假设 Cell 里有个 model 属性
45     self.titleLabel.frame = model.titleLabelFrame;
46     self.subtitleLabel.frame = model.subtitleLabelFrame;
47     self.iconImageView.frame = model.iconImageViewFrame;
48 }

```

```

45
46 // 在 Controller 里
47 - (UITableViewCell *)tableView:(UITableView *)tableView cellForRowAtIndexPath:
(NSIndexPath *)indexPath {
48     MyTableViewCell *cell = [tableView
dequeueReusableCellWithIdentifier:@"MyCell" forIndexPath:indexPath];
49
50     MyModel *model = self.dataArray[indexPath.row];
51     cell.model = model; // 把 model 传给 Cell
52
53     cell.titleLabel.text = model.title;
54     cell.subtitleLabel.text = model.subtitle;
55
56     // ✅ 优化: 使用 SDWebImage 异步加载和解码
57     [cell.imageView sd_setImageWithURL:[NSURL
URLWithString:model.iconImageUrl] placeholderImage:[UIImage
imageName:@"placeholder"]];
58
59     return cell;
60 }
61
62 - (CGFloat)tableView:(UITableView *)tableView heightForRowAtIndexPath:
(NSIndexPath *)indexPath {
63     // ✅ 优化: 直接返回 Model 里预计算好的高度
64     MyModel *model = self.dataArray[indexPath.row];
65     return model.cellHeight;
66 }

```

案例二：优化圆角图片（消除离屏渲染）

优化前：触发离屏渲染

Code block

```

1  UIImageView *imageView = [[UIImageView alloc] initWithFrame:CGRectMake(100,
100, 100, 100)];
2  imageView.image = [UIImage imageNamed:@"avatar"];
3  imageView.layer.cornerRadius = 50; // 直径的一半
4  imageView.layer.masksToBounds = YES; // 关键: 这个会触发离屏渲染
5  [self.view addSubview:imageView];

```

优化后：后台预处理图片

code block 创建一个后台队列用于图像处理

```
1 static dispatch_queue_t imageProcessingQueue = NULL;
2 static dispatch_once_t onceToken;
3 dispatch_once(&onceToken, ^{
4     imageProcessingQueue =
5     dispatch_queue_create("com.yourapp.image.processing", DISPATCH_QUEUE_SERIAL);
6 });
7
8 // 2. 定义一个处理圆角图片的函数
9 + (UIImage *)circleImage:(UIImage *)image withSize:(CGSize)size {
10     UIGraphicsBeginImageContextWithOptions(size, NO, [UIScreen
11     mainScreen].scale);
12     CGContextRef context = UIGraphicsGetCurrentContext();
13
14     // 创建一个圆形的剪切路径
15     UIBezierPath *path = [UIBezierPath bezierPathWithOvalInRect:CGRectMake(0,
16     0, size.width, size.height)];
17     [path addClip];
18
19     // 将原始图片绘制到上下文中
20     [image drawInRect:CGRectMake(0, 0, size.width, size.height)];
21
22     // 从上下文中获取新的图片
23     UIImage *roundedImage = UIGraphicsGetImageFromCurrentImageContext();
24     UIGraphicsEndImageContext();
25
26     return roundedImage;
27 }
28
29 // 3. 在需要设置图片的地方使用
30 UIImageView *imageView = [[UIImageView alloc] initWithFrame:CGRectMake(100,
31 100, 100, 100)];
32
33 // 将图片处理放到后台队列
34 dispatch_async(imageProcessingQueue, ^{
35     UIImage *originalImage = [UIImage imageNamed:@"avatar"];
36     UIImage *processedImage = [self circleImage:originalImage
37     withSize:CGSizeMake(100, 100)];
38
39     // 处理完成后, 回到主线程更新 UI
40     dispatch_async(dispatch_get_main_queue(), ^{
41         imageView.image = processedImage;
42     });
43 });
```

案例三：优化列表滚动流畅度（异步布局）

注意：异步布局（如使用 Texture/ASDK）涉及框架的使用，代码量较大。这里用一个简单的 NSOperationQueue 模拟其核心思想。

优化前：布局计算在主线程

Code block

```
1  - (UITableViewCell *)tableView:(UITableView *)tableView cellForRowAtIndexPath:
    (NSIndexPath *)indexPath {
2      MyTableViewCell *cell = [tableView
    dequeueReusableCellWithIdentifier:@"MyCell" forIndexPath:indexPath];
3
4      MyModel *model = self.dataArray[indexPath.row];
5
6      // ❌ 错误示范：在主线程计算复杂布局
7      CGSize textSize = [model.content
    boundingRectWithSize:CGSizeMake(SCREEN_WIDTH - 40, CGFLOAT_MAX)
8
    options:NSStrngDrawingUsesLineFragmentOrigin
9                                     attributes:@{NSFontAttributeName:
    [UIFont systemFontOfSize:14]}
10                                     context:nil].size;
11
12     // 根据 text 的高度，计算 cell 里其他元素的位置
13     // ... 更多计算 ...
14
15     cell.textLabel.text = model.content;
16     // ... 设置其他 UI ...
17
18     return cell;
19 }
```

优化后：布局计算在后台线程

Code block

```
1  // 1. 创建一个后台队列
2  @property (nonatomic, strong) NSOperationQueue *layoutQueue;
3
4  - (void)viewDidLoad {
5      [super viewDidLoad];
6      self.layoutQueue = [[NSOperationQueue alloc] init];
7      self.layoutQueue.maxConcurrentOperationCount = 2; // 控制并发数
8  }
```

```

9
10 // 2. 在 Model 里增加一个属性, 用于存储布局结果
11 @interface MyModel : NSObject
12 @property (nonatomic, strong) id layoutResult; // 存储布局计算结果的字典
13 @end
14
15 // 3. 在数据加载后, 启动后台布局计算
16 - (void)onDataReceived:(NSArray<MyModel *> *)data {
17     for (MyModel *model in data) {
18         NSBlockOperation *operation = [NSBlockOperation
19 blockOperationWithBlock:^(
20         // 在后台线程进行复杂计算
21         CGSize textSize = [model.content
22 boundingRectWithSize:CGSizeMake(SCREEN_WIDTH - 40, CGFLOAT_MAX)
23 options:NSStrngDrawingUsesLineFragmentOrigin
24 attributes:@{NSFontAttributeName: [UIFont systemFontOfSize:14]}
25 context:nil].size;
26
27         // ... 计算更多布局 ...
28
29         // 将计算结果存入 model
30         model.layoutResult = @{
31             @"textFrame" : [NSValue valueWithCGRect:CGRectMake(10, 10,
32 textSize.width, textSize.height)],
33             @"cellHeight" : @(10 + textSize.height + 20) // top + text +
34 bottom
35         };
36     }];
37
38     [self.layoutQueue addOperation:operation];
39 }
40
41 self.dataArray = data; // 保存数据
42 }
43
44 // 4. 在 Cell 里直接使用计算好的结果
45 - (UITableViewCell *)tableView:(UITableView *)tableView cellForRowAtIndexPath:
46 (NSIndexPath *)indexPath {
47     MyTableViewCell *cell = [tableView
48 dequeueReusableCellWithIdentifier:@"MyCell" forIndexPath:indexPath];
49
50     MyModel *model = self.dataArray[indexPath.row];
51
52     //  优化: 直接使用后台计算好的布局结果
53     if (model.layoutResult) {

```

```
48     NSDictionary *layout = model.layoutResult;
49     CGRect textFrame = [layout[@"textFrame"] CGRectValue];
50     cell.textLabel.frame = textFrame;
51     cell.textLabel.text = model.content;
52     // ... 设置其他 UI ...
53 }
54
55 return cell;
56 }
```

总结

这些代码示例展示了从“阻塞主线程”到“异步处理”、“预计算”、“后台处理”的优化思路，可直接应用到实际 iOS 项目中提升 UI 流畅度。

(注：文档部分内容可能由 AI 生成)